Assignment 8.2 – Identifying Security Risks and Strengthening Deployment Practices

Date: October 3, 2026
Student: Jonathan Cantu
Professor: Richard Krasso
Course: WEB430

Part I: NorthPeak Travel Systems Security Analysis

Assignment Instructions:

NorthPeak Travel Systems builds software used by travel agencies to manage bookings and itineraries. They recently automated their deployment process, but they are experiencing several security issues:

- Secrets stored directly in the repository
- Inconsistent access controls for deployment environments
- AI-generated code that introduced a vulnerability
- Manual overrides during deployment
- No threat modeling process
- Limited logging and monitoring

Leadership wants a clear assessment of their security posture and practical recommendations for improvement.

Required Action:

Part I:

Using the readings from this week, write a 2-3 page analysis (roughly 600-900 words) that addresses the following:

1). Identify Security Risks
Describe where NorthPeak is exposed to risk and why it matters.

2). Evaluate Secure Coding Practices
Explain how insecure code can move through the pipeline and what the team should do differently.

3). Assess Deployment Security
Identify weaknesses in the deployment process and how to address them.

4). Recommend Threat Modeling Practices
Provide a simple, practical approach the team can adopt.

5). Explain How AI Tools Should be Used Securely

Identify where AI can help and where oversight is needed.

How this prepares you for the Final Project (Week 9):
You will design an AI-augmented workflow. Understanding secure development and deployment practices is essential for designing a workflow that is both effective and safe.

NorthPeak Travel Systems has successfully automated portions of its deployment process, but automation without appropriate security controls can move vulnerabilities through a pipeline just as efficiently as it moves good code. The problems described in the scenario—repository secrets, inconsistent access controls, vulnerable AI-generated code, manual overrides, missing threat modeling, and limited monitoring—indicate that security has not yet been fully integrated into the development lifecycle. Krasso (2026) emphasizes that security should be incorporated throughout development and deployment rather than treated as a final check. NorthPeak should therefore move toward a DevSecOps model where security becomes part of normal Flow and Feedback.

1. Identify Security Risks

Storing secrets directly in the repository is one of NorthPeak's most immediate risks. Credentials, API keys, and deployment tokens committed to source control can remain accessible through repository history even after the visible value is removed. These credentials should instead be stored in an appropriate secrets-management system.

GitHub Actions, for example, supports secrets at the organization, repository, and environment levels. Environment-level secrets are particularly useful for separating staging and production credentials, while access policies can restrict which repositories receive organization-level secrets. GitHub also recommends granting credentials only the minimum permissions necessary (GitHub, 2026).

Inconsistent deployment access is another significant concern. Production access should follow least-privilege principles and clearly defined roles rather than depend on who happens to perform a deployment. Manual overrides are particularly concerning when they allow required controls to be bypassed without sufficient authorization, documentation, and auditing.

Limited logging and monitoring also reduce NorthPeak's ability to recognize or investigate security incidents. This is significant enough that OWASP (2025) identifies Security Logging and Alerting Failures among its current Top 10 web application security risks.

2. Evaluate Secure Coding Practices

The AI-generated vulnerability demonstrates why AI-generated code should be treated like any other untrusted code contribution until it has been reviewed and validated. AI can accelerate development, but it can also reproduce insecure patterns, misunderstand application context, or generate code that appears correct while introducing a vulnerability.

NorthPeak should require AI-assisted code to pass through the same pull request, peer review, automated testing, and security quality gates as human-written code. Developers should remain accountable for understanding and validating code before approving it.

Automated security scanning can strengthen this process. Static application security testing, dependency scanning, and software composition analysis can identify problems earlier in the pipeline. OWASP's 2025 Top 10 specifically identifies Software Supply Chain Failures as a major risk and recommends tracking dependencies and regularly scanning them for known vulnerabilities (OWASP, 2025).

From my experience, tools such as Snyk can be incorporated into CI/CD pipelines to make vulnerability detection part of the normal development feedback loop rather than a separate activity near release. A GitHub Actions workflow could run automated security checks when a pull request is created or updated and establish risk-based quality gates before the code can progress.

3. Assess Deployment Security

A secure deployment should be repeatable, authorized, traceable, and based on an artifact that has already passed required quality and security checks. NorthPeak currently undermines those characteristics through inconsistent access controls and manual overrides.

Deployment environments should have separate credentials and permissions, with production receiving stricter controls than development or staging. GitHub Actions environments and secrets could help implement this separation. GitHub also supports mechanisms such as short-lived authentication through OpenID Connect, which can reduce dependence on long-lived cloud credentials stored as secrets (GitHub, 2026).

Security gates should occur before deployment. Tests, vulnerability scans, and other required checks should fail the pipeline when findings exceed an established risk threshold. However, I would not recommend blindly blocking deployment for every warning. High or critical findings may justify stopping promotion, while lower-severity findings could enter a documented remediation process.

Manual overrides should be exceptional rather than routine. When an override is necessary, it should require authorization, record who approved it and why, and produce an audit trail. Otherwise, automation provides little protection if people can routinely bypass its controls.

4. Recommend Threat Modeling Practices

NorthPeak does not need an overly complicated threat-modeling program to begin receiving value from the practice. I would start with a lightweight process during planning and architecture discussions.

For significant features or infrastructure changes, the team should ask four basic questions:

1. What are we building or changing?
2. What assets, data, credentials, or services need protection?
3. What could go wrong or be abused?
4. What controls will reduce those risks?

The resulting threats and mitigations should become actionable backlog items rather than remain in a separate security document. Microsoft recommends integrating threat modeling into DevOps so

identified mitigations can become work items and security events can feed monitoring and incident response (Microsoft, 2026).

Threat models should also evolve with the application. A new external API, authentication mechanism, deployment process, or AI integration can change the threat surface and should trigger reconsideration of the existing model.

5. Explain How AI Tools Should Be Used Securely

AI can provide useful security assistance without becoming the security authority. It could summarize vulnerability scan results, identify suspicious coding patterns, help categorize security findings, analyze logs, or suggest possible mitigations. These uses reduce cognitive load and can help developers understand large amounts of security information more quickly.

However, NorthPeak should not automatically accept AI-generated fixes or allow an AI agent to bypass deployment controls. AI output should be reviewed against secure coding standards, application requirements, and actual security evidence.

The same principle should apply to AI-generated security alerts. AI may help identify patterns humans could miss, but significant actions—such as changing access controls, rotating production credentials, disabling services, or approving a production deployment—should retain appropriate human oversight.

NorthPeak's goal should not be maximum automation. It should be **trustworthy automation**. Security checks should be integrated early enough to provide useful Feedback, deployment permissions should follow least privilege, secrets should be properly managed, and threat modeling should identify risks before they become production vulnerabilities. AI can strengthen that process, but responsibility for the security of the system must remain with the development and operations teams.

References:

GitHub. (2026). "Secrets". GitHub Docs. https://docs.github.com/en/actions/concepts/security/secrets
Krasso, Richard (2026). "WEB 430 Weekly Reading: Week 8". Bellevue University.
Microsoft. (2026). "Integrating threat modeling with DevOps". Microsoft Learn. https://learn.microsoft.com/en-us/security/engineering/threat-modeling-with-dev-ops
OWASP Foundation. (2025). "OWASP Top 10". https://top10.owasp.org/2025/

Part II

For this part of the assignment, you will work with a small code snippet and a deployment configuration. You will use an AI-assisted tool to:

- Identify security risks
- Suggest improvements
- Evaluate the quality of the AI's recommendations

This gives you practical experience reviewing code and configuration files for security issues.

Code Snippet (Python):

```
import os
```

```python
def connect_to_db():
  username = "admin"

  password = "SuperSecret123"

  host = os.getenv("DB_HOST", "localhost")

  return f"Connecting to {host} with {username}/{password}"
```

Deployment Snippet (YAML)

```yaml
env:
  DB_HOST: production-db.internal
  API_KEY: "12345-ABCDE"
  LOG_LEVEL: debug
```

Steps:

1. Copy both snippets into an AI-assisted tool.
2. Ask the tool to identify security risks.
3. Ask the tool to suggest improvements.
4. Ask the tool to explain why the risks matter.
5. Review the output and reflect on what you notice.

What to Submit for the Hands-on Portion:

1. AI-generated list of security risks
2. AI-generated improvement suggestions
3. AI-generated explanations
4. A short reflection (8-10 sentences) addressing:
5. What the AI identified correctly
6. What it missed
7. Whether the suggestions were practical
8. How this connects to secure development and deployment

AI Response

🔒 Security Review: Python Code & YAML Configuration

🚨 Security Risks

Python Code Risks

- **Hardcoded credentials** (`username` and `password`) in source code
- **Sensitive data exposure** through connection string construction
- **No encryption** for database credentials
- **Credentials stored in plaintext** even in memory
- **Potential credential leakage** in logs or error messages

Deployment YAML Risks

- **Hardcoded API key** in version-controlled configuration

- **API key exposed** in deployment manifests
- **Insecure default log level** (`debug` might expose sensitive data)
- **No secret management system** for sensitive environment variables
- **Production credentials** in readable configuration files

🛠️ Suggested Improvements

For Python Code

- **Use environment variables** or secret management service for credentials:

```
username = os.getenv("DB_USERNAME")
password = os.getenv("DB_PASSWORD")
```

- **Implement credential validation** and secure handling
- **Use connection pooling** with encrypted credentials
- **Add credential masking** in logs and error messages
- **Consider using a secrets manager** (AWS Secrets Manager, HashiCorp Vault)

For Deployment Configuration

- **Replace hardcoded API key** with secret references:

```
env:
  API_KEY: "{{ .Secrets.apiKey }}"
```

- **Use Kubernetes Secrets** or similar orchestration secrets
- **Set appropriate log level** for production:

```
LOG_LEVEL: info  # or warn/error for production
```

- **Implement secret rotation** policies
- **Use encrypted secret stores** for sensitive data

⚠️ Why the Risks Matter

Credential Exposure Consequences

- **Unauthorized database access** leading to data breaches
- **API key misuse** enabling malicious API calls
- **Credential theft** from version control history
- **Lateral movement** within infrastructure if one service is compromised

Production Environment Impact

- **Regulatory violations** (GDPR, HIPAA, PCI-DSS) from exposed credentials
- **Reputational damage** from data leaks or service disruptions
- **Financial losses** from fraudulent activities or ransom demands
- **Operational downtime** during credential rotation after exposure

Logging Risks

- **Debug logs in production** can expose:
  - SQL queries with sensitive data
  - User information in stack traces
  - Internal system details useful to attackers

- Session tokens or authentication details

Configuration Management Issues

- **Secret sprawl** across multiple deployment files
- **Difficulty in rotation** when credentials are hardcoded
- **Audit trail gaps** for credential access and usage
- **Privilege escalation** through compromised service accounts

---

Key Principle: Always treat credentials and API keys as sensitive secrets that should never be stored in plaintext within code or configuration files. Use dedicated secret management solutions appropriate for your deployment environment.

Reflection

I thought DeepSeek did a good job identifying the most significant security problems in both the Python code and deployment configuration. In particular, it recognized not only that the database password and API key were hardcoded, but also that the function exposes the database credentials in its returned string, which could cause them to appear in logs or other output. Its recommendations to externalize secrets, rotate exposed credentials, use a least-privilege database account, and add automated secret scanning are practical improvements that could be incorporated into a CI/CD workflow. I also agreed with its concern about running production systems with debug logging enabled, although debug logging itself is not automatically a vulnerability unless sensitive information is actually being logged.

One area where I think the analysis overstated the risk was identifying the internal database hostname as a security exposure; an internal hostname is useful reconnaissance information, but it is not equivalent to exposing a password or API key. Likewise, the example API key looks weak, but without knowing how the real key is generated, DeepSeek cannot conclusively determine its entropy or strength. I particularly liked the recommendation to treat committed credentials as compromised and rotate them rather than assuming that simply removing them from the current source file resolves the problem.

This exercise reinforces the Week 8 principle that security should be incorporated throughout development and deployment, including source control, code review, secrets management, logging, and automated pipeline checks. It also demonstrates why AI can be useful for quickly identifying potential security problems while still requiring human judgment to evaluate the severity, context, and practicality of its recommendations.